

ESTRUTURAS DE DADOS E ALGORITMOS

Algoritmos de Correspondência de Strings

Aula 11 · encontrar uma agulha num palheiro de texto

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

Na Aula 3: qual era a complexidade de verificar, por força bruta, se um texto contém uma palavra?

2

O que faz uma função de hash? (Aula 10)

3

Para que serve o operador % (mod) nas tabelas de dispersão?

Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

Carrega em Ctrl + F e escreve uma palavra. O que faz o computador por trás?

1

Editores de texto

"procurar e substituir"

2

Motores de busca

*páginas que contêm as
palavras*

3

Bioinformática

*sequências de ADN:
GATTACA...*

4

Deteção de plágio

trechos repetidos entre textos

Um genoma humano tem cerca de 3 mil milhões de letras. Procurar à força bruta pode sair caro — hoje vemos como fazer melhor.

Do ingênuo ao engenhoso

1

O problema

Encontrar um padrão num texto.

2

Algoritmo ingênuo

Força bruta: testar todas as posições.

3

Rabin-Karp

Comparar "impressões digitais" (hashes).

4

Hash rolling

Atualizar o hash sem recalculá-lo tudo.

Pergunta-guia: como comparar um bloco de texto inteiro com uma única comparação?

1

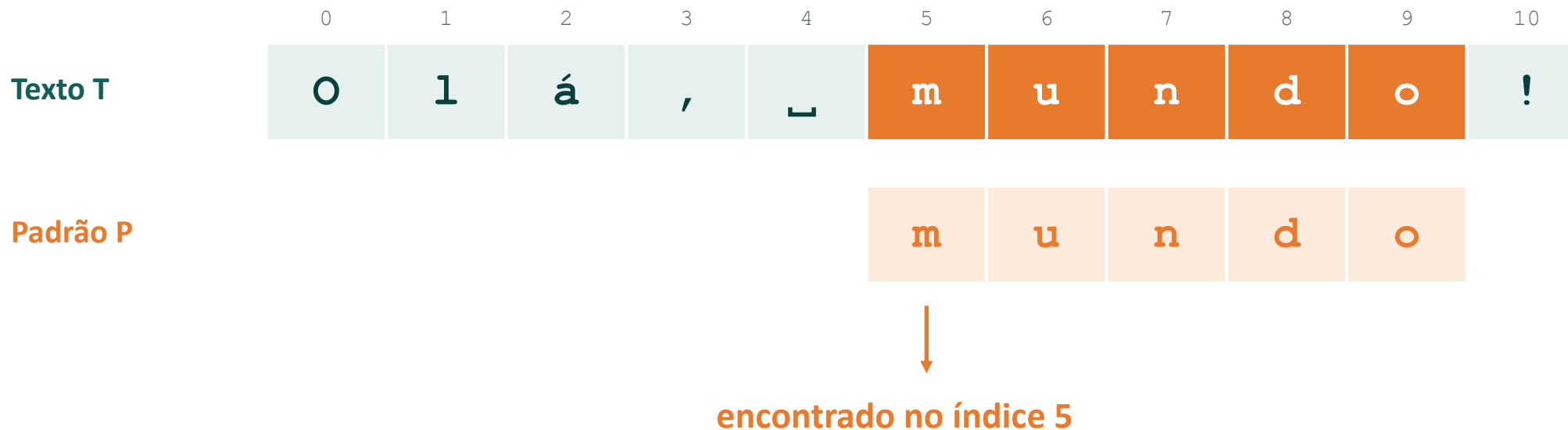
PARTE 1

0 problema

Encontrar uma string curta numa string longa.

Onde aparece o padrão no texto?

Encontrar as ocorrências de um **padrão P** (curto, comprimento m) dentro de um **texto T** (longo, comprimento n).

**n**

comprimento do texto (aqui 11)

m

comprimento do padrão (aqui 5)

2

PARTE 2

Algoritmo ingênuo

Força bruta: tentar em todas as posições.

Alinhar, comparar, deslizar

1 Alinhar

Colocar o padrão na primeira posição do texto.



2 Comparar

Comparar o padrão com o texto, carácter a carácter.



3 Registrar

Se todos os caracteres coincidirem, é uma correspondência.

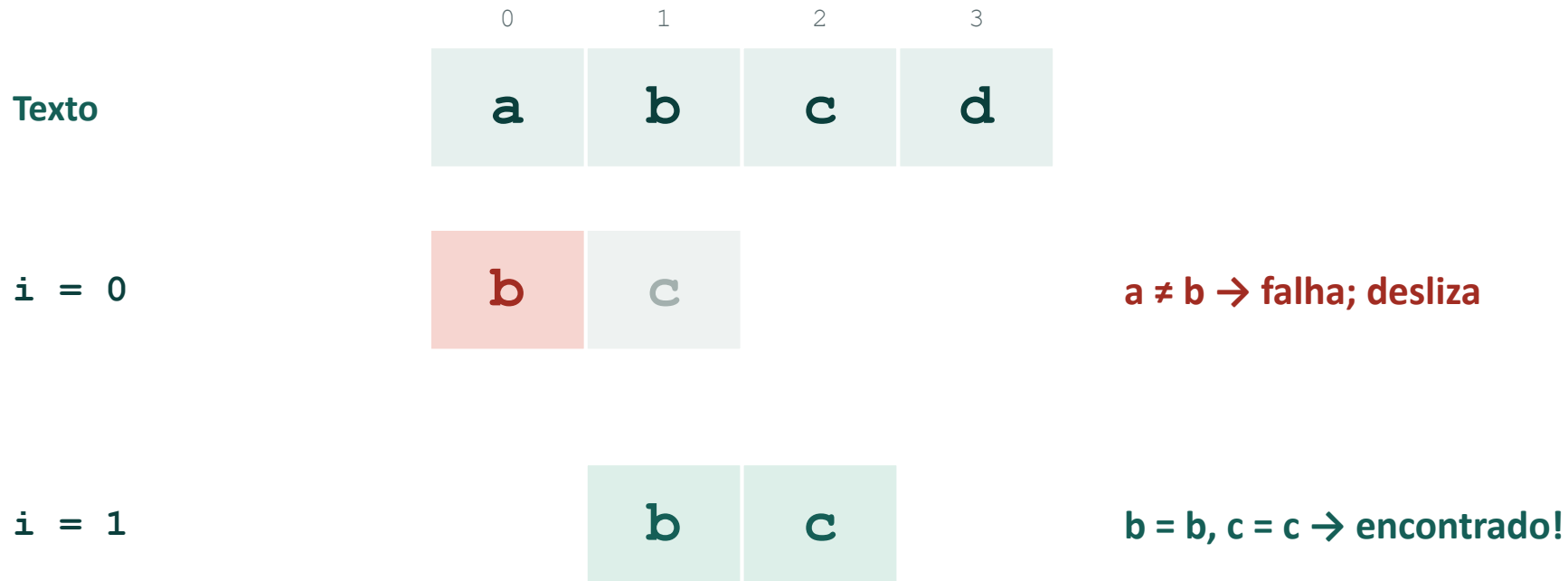


4 Deslizar

Avançar o padrão uma posição e repetir até ao fim.

```
int ingenuo(String T, String P) {  
    int n = T.length(), m = P.length();  
    for (int i = 0; i <= n - m; i++) {  
        int j = 0;  
        while (j < m && T.charAt(i+j) == P.charAt(j)) j++;  
        if (j == m) return i;    // encontrado  
    }  
    return -1;  
}
```

Procurar "bc" em "abcd"



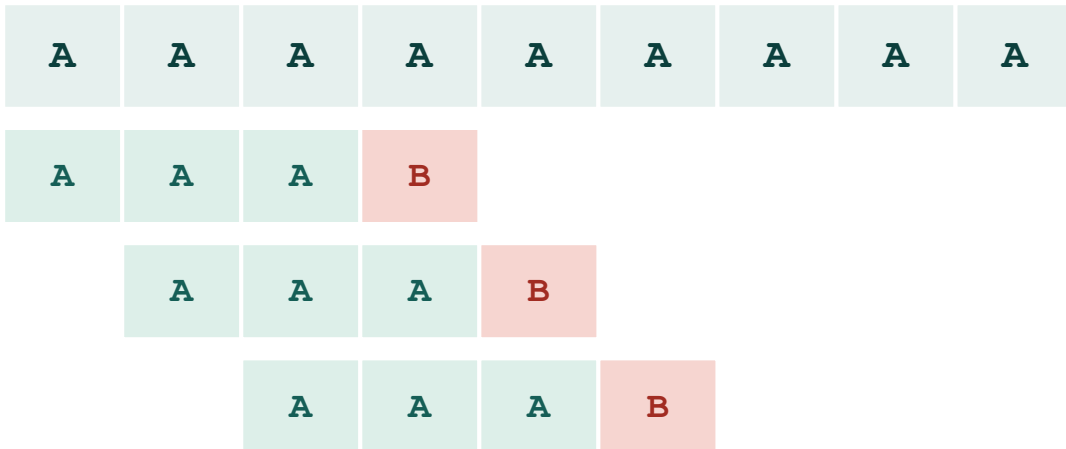
Resultado: índice 1, com 3 comparações de caracteres. Guarde este exemplo: vamos repeti-lo com o Rabin-Karp.

Simples, mas pode ficar lento

Pior caso: $O((n - m + 1) \cdot m)$

$n - m + 1$ posições $\times$ até m comparações em cada

O pior caso: falhar sempre no último carácter



Vantagens

- Muito simples e intuitivo
- Não precisa de memória extra
- Rápido em textos "normais"

Desvantagens

- Lento com textos e padrões grandes
- Volta a comparar caracteres já vistos

3

PARTE 3

Rabin-Karp

Comparar impressões digitais em vez de letras.

Comparar números em vez de palavras

Ingénuo

m	u	n	d	o
m	u	n	d	i

até m comparações de caracteres por posição

m = 5 → 5 comparações

Rabin-Karp

$h(\text{mundo}) = 59$

$h(\text{mundi}) = 53$

≠

uma só comparação de números

hashes diferentes → não coincide

Hashes iguais? Confirma-se letra a letra (pode ser coincidência).

Uma string lida como um número

$$\text{hash}(S) = (S[0] \cdot d^{m-1} + S[1] \cdot d^{m-2} + \dots + S[m-1] \cdot d^0) \bmod q$$

d

número de caracteres do alfabeto (256 no ASCII)

q

um número primo, que mantém o hash pequeno

S[i]

o código do carácter (ex.: a = 97, b = 98)

Cada carácter é um "algarismo" na base d.

Já conhece esta ideia

Ler "123" em base 10:

$$1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0 = 123$$

Ler "bc" em base 256:

$$98 \cdot 256 + 99 = 25187$$

Calcular, deslizar, comparar, confirmar

- 1 Calcular** o hash do padrão e o da primeira janela do texto.
- 2 Deslizar** a janela uma posição de cada vez.
- 3 Atualizar** o hash da nova janela com hash rolling — sem recomeçar do zero.
- 4 Confirmar** se os hashes forem iguais, comparar letra a letra (colisões!).

No fim: registrar as correspondências encontradas.

Tirar um carácter, pôr outro

Como passar de 123 para 234, em base 10?

123

- 1·100

→ 23

tirar o que sai

× 10

→ 230

subir uma casa

+ 4

→ 234

pôr o que entra

A fórmula geral

$$\text{hash}_{\text{next}} = (d \cdot (\text{hash}_{\text{atual}} - T[i] \cdot h) + T[i+m]) \bmod q$$

$$h = d^{m-1} \bmod q \quad (\text{o "peso" do carácter mais à esquerda})$$

$T[i]$

carácter que sai

$T[i+m]$

carácter que entra

Porque é rápido? Cada nova janela custa um punhado de operações — $O(1)$ — em vez de recalcular os m caracteres. **Hash negativo?** Soma-se q .

4

PARTE 4

Exemplo resolvido

Procurar "bc" em "abcd", com números.

Texto "abcd", padrão "bc", $d = 256$, $q = 101$

0	1	2	3
a	b	c	d
97	98	99	100

janela 0

bc

padrão ($m = 2$)

$d = 256$ alfabeto ASCII
 $q = 101$ número primo

1 Hash do padrão "bc"

$$(98 \cdot 256 + 99) \bmod 101$$

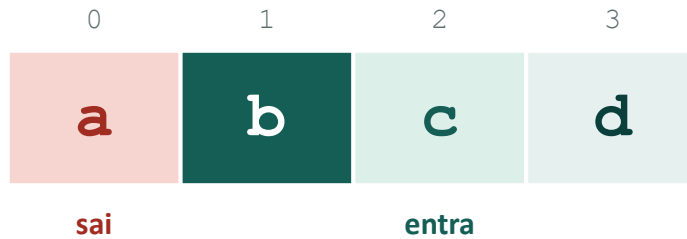
$$25187 \bmod 101 = 38$$

2 Hash da 1.ª janela "ab"

$$(97 \cdot 256 + 98) \bmod 101$$

$$24930 \bmod 101 = 84$$

Deslizar a janela de "ab" para "bc"



hash atual 84 (ab)

T[i] a = 97 carácter que sai

T[i+m] c = 99 carácter que entra

h 256 mod 101 = 54

$$\begin{aligned} \text{hash}_{\text{next}} &= (256 \cdot (84 - 97 \cdot 54) + 99) \bmod 101 \\ &= -1 \ 319 \ 325 \bmod 101 = \mathbf{38} \end{aligned}$$

Atenção em Java: $-1319325 \% 101$ dá -63 ! O resto matemático é $38 = -63 + 101$. Use `Math.floorMod` ou some q.

Hash igual? Confirmar letra a letra

Janela "bc"

hash = 38

=

padrão = 38

b	c
b	c

b = b ✓ c = c ✓

Encontrado no índice 1

Porque é preciso confirmar?

Com $q = 101$ só há 101 hashes possíveis. Estas strings também têm hash 38:

kv

mo

oh

qa

zt

Hash igual = talvez.

Hash diferente = de certeza que não.

Um q primo grande torna as colisões raras.

Verdadeiro ou falso?

Se os hashes forem iguais, o padrão está de certeza naquela posição.

Falso

pode ser uma colisão: confirma-se letra a letra

Se os hashes forem diferentes, o padrão não está naquela posição.

Verdadeiro

strings iguais têm sempre o mesmo hash

O hash rolling recalcula a janela inteira a cada passo.

Falso

só tira um carácter e acrescenta outro: $O(1)$

Tape a coluna da direita e peça a resposta antes de a mostrar.

Rápido em média, mas não sempre

	Ingénuo	Rabin-Karp
Caso médio	rápido em textos normais	$O(n + m)$
Pior caso	$O(n \cdot m)$	$O(n \cdot m)$
Custo de passar à janela seguinte	recomeça do zero	$O(1)$ com hash rolling
Memória extra	$O(1)$	$O(1)$
Ponto forte	simplicidade	procurar vários padrões de uma vez

Nota: $O(n + m)$ é o tempo esperado. No pior caso (muitas colisões), o Rabin-Karp volta a $O(n \cdot m)$. Existem algoritmos garantidamente lineares, como o KMP.

Agora é consigo



Algoritmo ingênuo



Implemente em Java o algoritmo ingênuo. Devolve o índice onde o padrão começa, ou -1 .

Teste: `"abcd" / "bc" → 1`; `"abcd" / "xy" → -1`.



RK em papel



Aplique o Rabin-Karp para encontrar "llo" em "Hello, world!". Mostre todos os cálculos.

Use $d = 256$ e $q = 101$. Pista: $m = 3$, logo $h = 256^2 \bmod 101$.



Rabin-Karp em Java



Implemente em Java o Rabin-Karp. Devolve o índice onde o padrão começa, ou -1 .

Não se esqueça de confirmar letra a letra e de usar `Math.floorMod`.

Três ideias a reter

1

Ingénuo

Testa todas as posições, letra a letra.
Simples; pior caso $O(n \cdot m)$.

2

Rabin-Karp

Compara hashes: diferente $\rightarrow$ não;
igual $\rightarrow$ confirmar letra a letra.

3

Hash rolling

Tira o carácter que sai, põe o que
entra: $O(1)$ por janela.

Força bruta (Aula 3) + funções de hash (Aula 10) = Rabin-Karp.

Bibliografia

1

Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms* (3.^a ed.). Pearson.

2

Karp, R. M., & Rabin, M. O. (1987). Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2), 249–260.

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt